Coeficiente de Variação com OpenMP

Exercício Final – Análise de Dados com OpenMP · Ciência da Computação – IESB
Prof. Rodrigo · Aluna: Paula Ribeiro Moreira de Souza

1. Visão geral

O programa `cv_openmp.c` lê o arquivo `dados.txt` (1000 linhas, cada uma com estatura em cm e peso em kg separados por espaço), agrupa cada variável em intervalos de classe (8 cm para estatura e 4 kg para peso) e calcula a média ponderada, o desvio padrão populacional e o coeficiente de variação. O cálculo foi dividido em quatro etapas, cada uma em uma função com seu próprio laço: (1) mínimo e máximo, (2) distribuição de frequências, (3) média ponderada e (4) desvio padrão. A primeira classe começa no piso do menor valor observado, e o número de classes é $k = \blacksquare(\text{máx} - \lim_{inf}) / h\blacksquare + 1$, de modo que todos os dados ficam dentro de alguma classe.

2. Partes paralelizadas e por quê

Todos os laços usam `#pragma omp parallel for` com uma cláusula `if(paralelo)`. Assim, o **mesmo código** roda nas versões sequencial (`paralelo = 0`, sem criar equipe de threads) e paralela, o que torna a comparação de tempo justa. Em todos os casos a sincronização é feita por **redução** (`reduction`): cada thread acumula em uma variável privada e o OpenMP combina os resultados no final, evitando condições de corrida sem usar `critical` ou `atomic`, que serializariam o acesso.

Etapa	Laço sobre	Diretiva OpenMP	Justificativa
Mínimo e máximo	N dados	`reduction(min:lo)` `reduction(max:hi)`	Iterações independentes; cada thread acha o mín/máx do seu bloco.
Frequências	N dados	`reduction(+:freq[:k])`	Etapa mais custosa. Redução de vetor: cada thread conta em uma cópia privada de freq[].
Média ponderada	k classes	`reduction(+:soma_xf, soma_f)`	Soma de $x_i \cdot f_i$ e de f_i, conforme a fórmula (1).
Desvio padrão	k classes	`reduction(+:soma)`	Soma de $f_i(x_i - \mu)^2$, conforme a fórmula (2).

Os laços de média e desvio padrão percorrem apenas as *k* classes (7 para estatura e 18 para peso), não as 1000 pessoas. Eles foram paralelizados porque o enunciado pede, mas com tão poucas iterações o ganho é desprezível; o trabalho que realmente escala com o tamanho da entrada está nos laços de mínimo/máximo e de frequências. O trecho principal da contagem de frequências é:

```
#pragma omp parallel for if(paralelo) reduction(+:freq[:k])
for (long i = 0; i < n; i++) {
    int c = (int)((v[i] - lim_inf) / h);   /* classe do dado i  */
    if (c >= k) c = k - 1;
    freq[c]++;                             /* cópia privada     */
}
```

3. Resultados

Variável	h	Classes (k)	Média μ	Desvio σ	CV
Estatura	8 cm	7	170,38 cm	9,16 cm	**5,37 %**
Peso	4 kg	18	70,07 kg	12,67 kg	**18,08 %**

O peso tem variabilidade relativa cerca de 3,4 vezes maior que a estatura. Isso é esperado: a estatura das pessoas varia pouco em torno da média, enquanto o peso depende de vários fatores além da altura. Como conferência, o CV calculado direto dos dados brutos (sem agrupamento) é 5,30 % para estatura e 18,08 % para peso, muito próximo dos valores agrupados; a diferença vem da aproximação de cada dado pelo ponto médio da sua classe.

Distribuições de frequências usadas no cálculo:

Classe (cm)	xi	fi
[145 ; 153)	149	22
[153 ; 161)	157	122
[161 ; 169)	165	302
[169 ; 177)	173	334
[177 ; 185)	181	162
[185 ; 193)	189	51
[193 ; 201)	197	7
	Total	1000

Classe (kg)	xi	fi
[42 ; 46)	44	27
[46 ; 50)	48	27
[50 ; 54)	52	45
[54 ; 58)	56	73
[58 ; 62)	60	111
[62 ; 66)	64	107
[66 ; 70)	68	117
[70 ; 74)	72	117
[74 ; 78)	76	112
[78 ; 82)	80	86
[82 ; 86)	84	70
[86 ; 90)	88	47
[90 ; 94)	92	25
[94 ; 98)	96	16
[98 ; 102)	100	15
[102 ; 106)	104	4
[106 ; 110)	108	0
[110 ; 114)	112	1

4. Análise de desempenho

O tempo foi medido com `omp_get_wtime()`. Como uma execução com N = 1000 leva poucos microssegundos, o programa repete o cálculo completo das duas variáveis 2000 vezes e reporta a média, após uma execução de aquecimento. Para observar o comportamento em escala, o mesmo código foi executado também com 10 milhões de registros gerados em memória (`./cv_openmp --bench 10000000`). Os testes foram executados em um GitHub Codespace com 2 vCPUs, isto é, 2 threads OpenMP disponíveis.

Versão	Threads	N = 1000 (ms)	Speedup	N = 10 mi (ms)	Speedup
Sequencial	1	0,0150	1,00×	49,998	1,00×
Paralela	1	0,0122	1,23×	49,466	1,01×
Paralela	2	0,0247	0,61×	49,340	1,01×

N = 1000. Com 2 threads, a versão paralela levou cerca de 1,6 vez o tempo da sequencial (speedup de 0,61×). O trabalho total é minúsculo (alguns milhares de operações), e o custo de criar e sincronizar a equipe de threads e combinar as reduções a cada região paralela supera o próprio cálculo. O resultado da versão paralela com 1 thread (1,23×) não indica ganho real: com tempos da ordem de microssegundos, pequenas variações de cache e de agendamento do sistema pesam muito na medição. Para esse tamanho de entrada, a versão sequencial é a melhor escolha.

N = 10 milhões. Com volume grande, o overhead do OpenMP deixa de pesar: as três configurações ficaram praticamente empatadas (cerca de 49–50 ms), em vez de a versão paralela ficar mais lenta. Ainda assim, 2 threads não reduziram o tempo pela metade. A explicação mais provável é o ambiente: em máquinas virtuais de nuvem, como o Codespace, cada vCPU costuma ser uma thread de hardware (hyper-threading), e 2 vCPUs podem corresponder a um único núcleo físico. Nesse caso as duas threads disputam as mesmas unidades de execução e o mesmo acesso à memória. Além disso, os laços sobre os dados fazem pouca conta por elemento lido, então tendem a ser limitados pela largura de banda da memória, que também é compartilhada. Em uma máquina com vários núcleos físicos, a expectativa é que o speedup desses laços cresça com o número de núcleos, limitado pela parte sequencial do programa (geração ou leitura dos dados e laços sobre as poucas classes), como prevê a Lei de Amdahl.

Conclusão. O programa produz os mesmos resultados nas versões sequencial e paralela e está corretamente paralelizado com reduções. O ganho de desempenho do OpenMP depende do volume de dados e do hardware: para as 1000 pessoas do exercício o overhead supera o benefício, e mesmo com 10 milhões de registros o ganho só aparece quando há núcleos físicos suficientes para executar as threads de fato em paralelo.